Payment status trigger: assumptions

Assumptions

#	Assumption	Basis	Impact if wrong
1	The trigger is a **recorded** status change into `overdue` (from any status, or none), and the invoice is still overdue when processed.	Inference: the brief asks for a status that "has just changed". The seed's overdue row lands at 00:05 the day after the due date, so a status job exists upstream.	If the business wants the calendar to decide, 1003 would get a link. The rule is one function (`evaluateInvoice`).
2	"Just changed" means **not yet handled**, not "recent". Each history row is handled once, with the outcome stored per row.	Inference: 1002 went overdue three months before "today". A time window would never fire for it.	With a freshness window, an old backlog would be skipped on purpose. That's a filter added to the run.
3	Money settles via invoice → park → legal entity → that entity's account **in the invoice currency**. Zero or several matches is an error, never a guess.	Inference from the schema: accounts belong to entities and carry a currency, and nothing marks a default account.	If a default-account rule exists, the ambiguity error becomes a lookup. The error path stays useful for "no account".
4	When the status snapshot and the latest history row disagree, nothing triggers.	Inference: either source may be stale, and sending money requests on stale data is worse than a delay.	Links wait until the data is fixed. The invoice is shown as "inconsistent".
5	**One payment link per invoice, enforced by the database**. A later re-entry into overdue reuses it.	Fact (Stripe API reference): Payment Links have no expiry field. Deactivation (`active: false`) exists.	A deactivated link would be reused. Fix: "one *active* link per invoice" (next steps).
6	The stub is a **Payment Link**, although its inputs look like a Checkout Session.	Fact (Stripe API reference): `POST /v1/payment_links` has no `customer`/`customer_email`. Checkout Sessions do, but expire 30 min–24 h after creation.	If the real wrapper creates sessions, a stored "link" dies within a day. It would have to mint a session per click instead.
7	The stub gets a second argument: an **idempotency key** per invoice, sent as Stripe's `Idempotency-Key`. Retries reuse the parameters first claimed.	Fact (Stripe docs): keys are kept for at least 24 h and may then be pruned; a pruned key creates a new object; the same key with different parameters errors.	Without it, a retry after a timeout can create a second link. After pruning, only our database prevents duplicates, which is why it is the long-term guard.
8	Same-key retries only help after timeouts and 409s.	Fact (Stripe docs): results are stored once execution starts, **including 400 and 500 errors**, and replayed for the same key. Only validation failures and 409 conflicts aren't stored.	After a stored 500 the trigger would retry into the same error. The real wrapper must classify errors and check whether the link exists (by metadata) before switching to a new key.
9	A claim lease of 2 minutes is longer than any Stripe call will take.	Guess (the mock has no timeout).	If a call can outlast the lease, another run takes over and both call Stripe. Within 24 h the shared key makes Stripe return one link; after that it may not. The lease must exceed the HTTP timeout.
10	A null `stripeCustomerId` is passed as null; Stripe creates or matches the customer from the email at checkout.	Fact plus inference: Payment Links have `customer_creation` (`always` / `if_required`).	Duplicate Stripe customers per email. Fix: create the customer first and store its id.
11	**D1 instead of static JSON or sql.js** in the browser.	Inference: "rerun must not duplicate" needs state that survives reloads, tabs and concurrent users. The trigger is a backend job (cron or event), not page logic.	With browser-only state, idempotency would be per tab, and two tabs would create two links. The logic itself is pure (`packages/domain`) and runs anywhere.
12	No database transactions; every guarantee rests on single atomic statements.	Fact (D1 driver source and Cloudflare docs): no interactive transactions, statements auto-commit, `batch()` is atomic.	None if D1 gained transactions. The design avoids needing them.
13	After a deploy, old and new code briefly run against the same database.	Fact (observed here): the old and new Worker were both answering for 10–30 s or more after deploys.	A migration that breaks the previous version breaks live traffic. Migrations must be expand-then-contract. The e2e waits for 5 consecutive answers from the new version.
14	A due date is past from 00:00 UTC the next day.	Guess. Parks are in DE/FR (UTC+1/+2).	The "past due" flag is off by a few hours. It never affects triggering.

Trigger condition

Chosen: a recorded history row that moves an invoice **into `overdue`**, not yet handled, with the invoice still `overdue` when the link is claimed.

The candidates in the seed, and why each lost:

- **Invoice created (`NULL → open`)**: rows 1, 3, 5, 6. A new invoice isn't a collection problem. Nudging every customer on day one is a billing feature, not dunning.
- **Due date passed**: 1001, 1003 and 1004 are all past due. 1001 is paid and 1004 is void, so that rule would chase paid and cancelled money. **1003** is the interesting one: past due, still `open`, with no overdue row. Either the status job lags or someone held it back deliberately (a payment plan, a dispute). The UI flags it instead of acting on it.
- **Snapshot `status = overdue`**: correct for 1002, but it carries no "just changed" signal. Without a ledger of handled rows it would fire on every run.

So 1002 (row 4, `open → overdue` on 2026-07-16) is the only trigger in the seed. A "mark overdue" demo control shows 1003 triggering once its status job records the change.

Routing logic

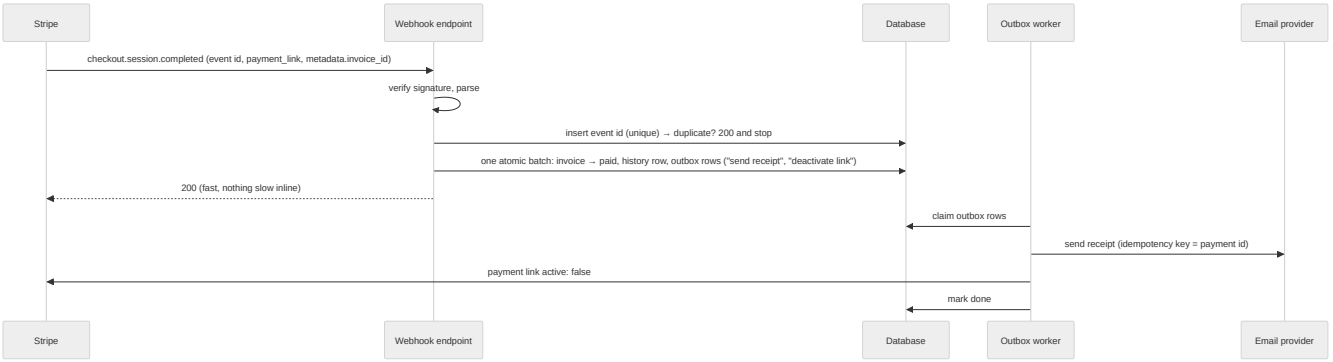
`invoice.park_id → parks.legal_entity_id → bank_accounts (legal_entity_id, currency = invoice.currency).`

What made it non-obvious:

- **Two parks in Munich belong to different legal entities.** Customer 100 has 1001 at München Nord and 1002 at München Süd. Routing by customer or city sends 1002's money to the wrong company. It must go to **Süd GmbH, account 2 (Commerzbank)**.
- **In the seed, bank account ids equal legal entity ids (1, 2, 3).** Passing the entity id as `settlementBankAccountId` would pass every seed-based test. It did in this project: a deliberate swap still passed all 20 tests at that point. A fixture with distinct ids now catches it.
- **The schema allows several accounts per entity.** Matching the invoice currency selects one. No match or several matches fails loudly, and the change is retried once the data is fixed.

Design note: the two non-implemented triggers

Both hang off one event: Stripe telling us money arrived. So the webhook is built first, and the receipt is a consequence of the status change it records.



1. Stripe webhook success → update invoice status

- **Trigger:** `checkout.session.completed` when the session's `payment_status` is `paid`, and `checkout.session.async_payment_succeeded` for delayed payment methods, which complete the session unpaid and settle later. Stripe recommends `checkout.session.completed` for Payment Links fulfilment. The invoice is found via the session's `payment_link` id (we store it per invoice), cross-checked against `metadata.invoice_id` (link metadata is copied to its sessions). Never match by amount or email.
- **What can go wrong, and the guard for each:**
  - **Duplicate delivery** (Stripe states it happens; it retries for up to 3 days). → Store `event.id` under a unique constraint and skip a repeat. Also skip if the invoice is already paid.
  - **Out-of-order events** (Stripe does not guarantee order). → Never let an event move an invoice backwards: `UPDATE … SET status='paid' WHERE id=? AND status IN ('open','overdue')`. A late "failed" event after "succeeded" changes nothing.

    - **Races with our own trigger** (paid while a link is being created). → The trigger already re-checks `overdue` inside its claim. On paid, an outbox row deactivates the invoice's link (`active: false`) so it can't be paid twice. Today nothing deactivates links: this is the top next step.
    - **Forged calls.** → Verify the `Stripe-Signature` header with the endpoint secret before reading anything.
    - **Slow handler → Stripe retries → more duplicates.** → Do one atomic D1 `batch()` (D1 has no interactive transactions; a batch is one), return 200, and push every external call to the outbox.
    - **Partial payments, refunds, disputes.** → Separate events (`charge.refunded`, `charge.dispute.created`) feeding their own status changes. Out of scope here, and flagged rather than ignored.
    - **Missed events** (endpoint down for more than 3 days). → A daily reconciliation lists the last day's paid sessions from the API and applies the same idempotent update.

2. Receipt of payment to the customer

- **Trigger:** the recorded `→ paid` history row, not the webhook directly. It's the same "handle each new history row once" pattern as the payment-link trigger, so a payment recorded by any path (webhook, bank transfer, manual) gets exactly one receipt.
- **What can go wrong, and the guard for each:**
    - **Double receipt** from duplicate webhooks or retried jobs. → One outbox row per payment, unique on the history row id. The email send uses an idempotency key (payment id) if the provider supports one.
    - **Receipt for money we didn't get** (async methods, a later dispute). → Send only on `paid`, not on `processing`. A refund sends a separate correction, never a silent edit.
    - **Email provider down or slow.** → Retry from the outbox with backoff. An outbox row is written in the same batch as the status change, so a crash cannot lose or duplicate it. Alert when a row stays unsent.
    - **Wrong legal entity on the receipt.** → Reuse the same routing as the link (invoice → park → legal entity). The receipt names the entity that received the money, which matters for invoices and tax.
    - **Missing or wrong email** (no Stripe customer, as for 1003's customer). → Use the invoice customer's email. If Stripe's automatic receipts are also switched on (a Dashboard setting, sent only for successful payments), the customer gets two receipts: pick one source.

Out of scope / what I'd do next

In priority order. Nothing here was rushed in; each item is either a production concern beyond a 3-hour take-home or depends on something the brief left out of scope.

1. **Deactivate links on paid/void, and allow one *active* link per invoice.** Today nothing deactivates a link, so a paid invoice's link stays payable (double-payment risk), and a re-triggered invoice would reuse a dead link.
    - Add a `generation` column and enforce uniqueness only for pending/created links (a partial unique index). Deactivated rows stay as history.
    - Use the key `payment-link:invoice:<id>:<generation>`. Reuse the active link, otherwise claim the next generation.
    - Not built: it needs the webhook trigger (design note), which is out of scope.
2. **Real Stripe wrapper.**
    - Payment Link versus Checkout Session decision (assumption 6).
    - Classify errors, and reconcile by metadata before switching keys after a stored 500 (assumption 8).
    - An HTTP timeout shorter than the claim lease (assumption 9).
    - Lowercase currency, and `metadata.invoice_id` on the link.
3. **Authentication and data exposure.** The demo is public by design. Anyone can read customer emails and IBANs, and any website can `POST /api/runs` or reset the demo (checked). Put the app behind Cloudflare Access or a token, mask emails and IBANs, and drop the demo endpoints outside demo stages.
4. **Scale-out.** Measured: about 5 D1 queries per newly overdue invoice, against D1's limit of 1,000 per invocation (50 on free). A cap of 100 invoices per run keeps each run safe and later runs continue the backlog. For real volume:
    - A Cron Trigger and a queue (one message per invoice).
    - Select new rows in SQL instead of loading whole tables.
    - Indexes on `payment_status_history(invoice_id)`.
    - Fewer round trips per run (batches) and Smart Placement. A run makes about 20 sequential D1 queries: 0.3 s from Europe, but from a US CI runner it was slow enough to flake one e2e spec once (the waits are fixed; the latency isn't).
5. **Rollouts.** Old and new Workers serve side by side for 10–30 s or more after a deploy. Write migrations as expand-then-contract, and consider gradual deployments with version checks.
6. **Smaller items.**
    - Due dates in each park's local time zone.
    - Alerting when an invoice stays inconsistent or unroutable for N runs.

- A test for a run that dies mid-way on real D1. Resumability is designed in (claims, leases, per-row decisions) but only tested in SQLite.